Développement Web:
Javascript & PHP

PARTIE 2: PHP

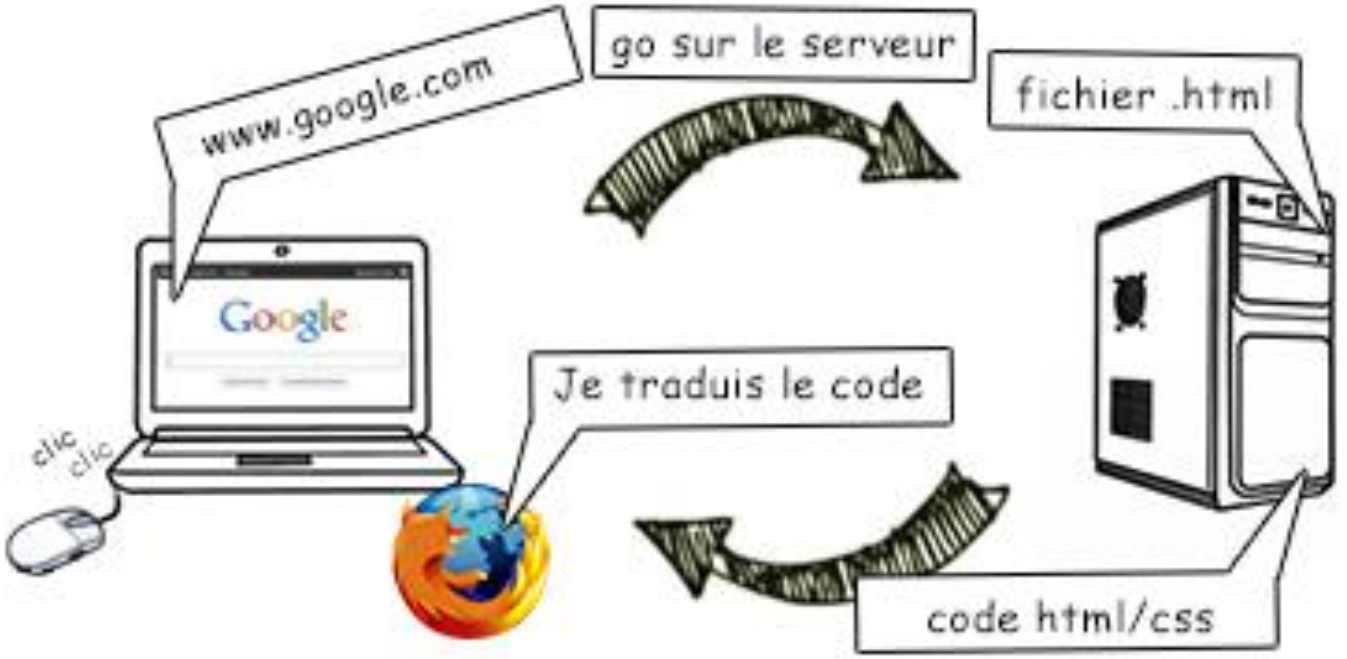
LES BASES DU LANGAGE PHP

PLAN

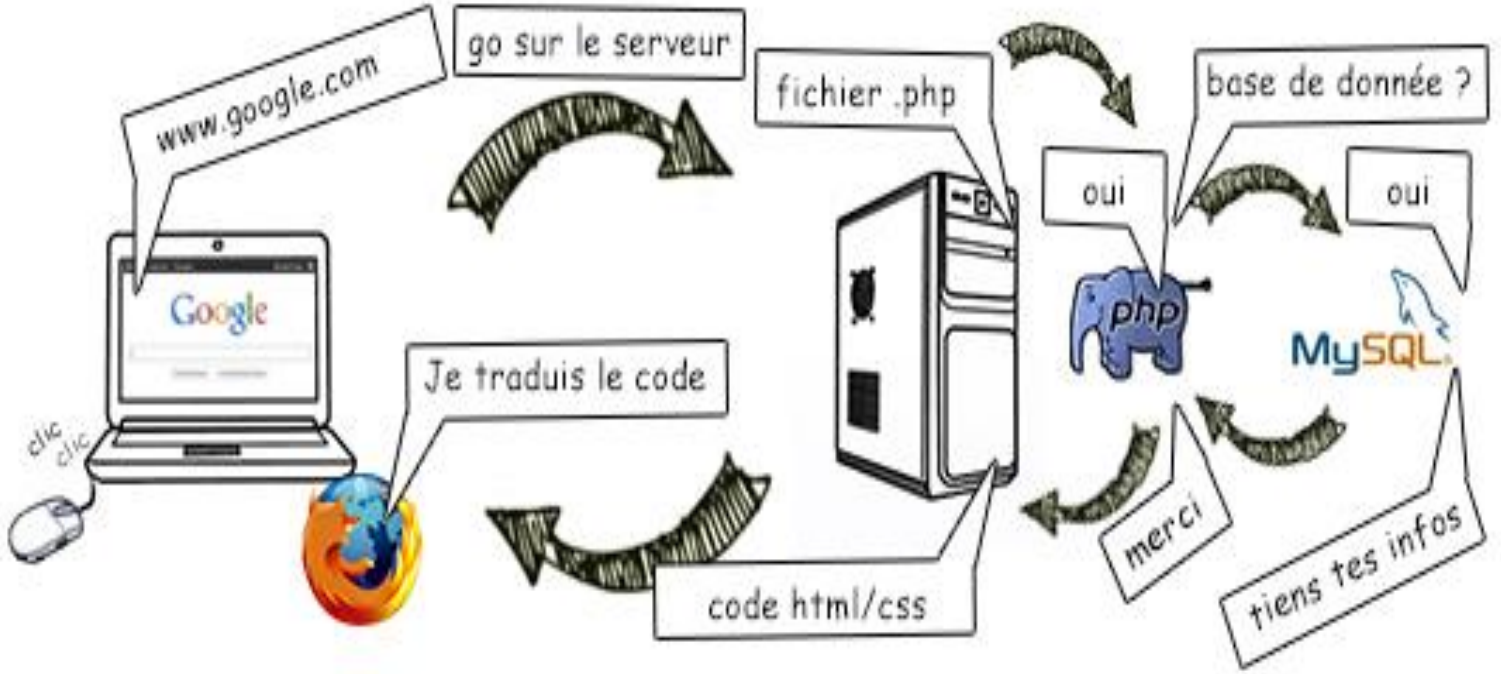
- Introduction
- Les variables et les constantes
- Les types de données
- Les opérateurs
- Les structures de contrôle

Introduction

Page web Statique Vs Dynamique



Page web Statique Vs Dynamique



Un peu d'histoire

- Histoire et Origine
  - PHP : **P**HP Hypertext **P**re**P**rocessor
  - Première version de PHP a été mis au point par Rasmus Lerdorf en 1994
    - Version appelée à l'époque **P**ersonal **H**ome **P**ages
    - Pour conserver la trace des utilisateurs venant consulter son CV sur son site, grâce à l'accès à une base de données par l'intermédiaire de requêtes SQL
  - La version 3.0 de PHP fut disponible le 6 juin 1998
  - A la fin de l'année 1999, une version bêta de PHP, baptisée PHP4 est apparue
  - En 2001 cinq millions de domaines utilisent PHP
    - trois fois plus que l'année 2000

PHP : C'est QUOI ?

- Un langage de scripts permettant la création d'applications Web
- Indépendant de la plate-forme utilisée
- La syntaxe du langage provient du langage C, Perl et Java.
- Ses principaux atouts sont:
  - La gratuité et la disponibilité du code source
  - La simplicité d'écriture de scripts
  - La possibilité d'inclure le script PHP au sein d'une page HTML
  - La simplicité d'interfaçage avec des bases de données
  - L'intégration au sein de nombreux serveurs web (Apache, Microsoft IIS, ...)
    - L'exécution des scripts PHP est déléguée à un composant indépendant installé sur e serveur web, souvent appelé moteur PHP

PHP: Exemple

```
<html >
  <head >
<title>Test de page PHP</title>
  </head >
  <body >
    <h2>Page de test</h2>
    <?php
      // Ceci est du code PHP
      $d = date("d/m/Y");    # Variable PHP + fonction
      $h = date("H:i:s");
      /* Affichage */
      echo "Nous sommes le <strong>$d</strong> et il est
                           <strong>$h</strong>";
    ?>
</body >
</html>
```

Commentaires

Intégration PHP et HTML

- Les scripts PHP sont intégrés dans le code d'un document HTML
- Les pages Web contenant des scripts PHP sont enregistrés avec l'extension **.php**
- L'intégration nécessite l'utilisation de balises:
  - `<?  ligne de code PHP  ?>`
  - **`<?php  ligne de code PHP  ?>`**
    - La méthode la plus utilisée
  - `<script language="php">  ligne de code PHP  </script>`
    - Moins utilisée

Intégration PHP et HTML

- Les scripts PHP peuvent être écrits:
  - Directement dans le fichier HTML
  - Dans des fichiers externes qui seront ensuite incorporés dans le document HTML

index.php

```
<html>
<head>
<title> Fichier d'appel </title>
</head>
<body>
<?php
$salut = " BONJOUR" ;
include "information.inc" ;
?>
</body>
</html>
```

information.inc

```
<?php
$chaine=$salut. " , C'est PHP "  ;
echo "$chaine";

?>
```

Intégration PHP et HTML

- Dans des fichiers externes qui seront ensuite incorporés dans le document HTML

Fonction	Description
`include('nomFichier ')`	Lors de l'exécution, cette fonction est remplacée par tout le contenu du fichier passé en paramètre. En cas d'erreur, par exemple le fichier n'est pas trouvé, elle génère une alerte (warning) et l'exécution du script continue.
`require('nomFichier ')`	Même comportement que « include ». Par contre, en cas d'erreur, elle provoque une erreur fatale et arrête l'exécution du script.
`include_once('nomFichier ')` `require_once('nomFichier ')`	Même comportement que les fonctions précédentes, avec la différence suivante: elles sont exécutées une seule fois.

Les variables

Les variables

- Pas de déclaration préalable
- Leur type est automatiquement défini par leur valeur
- Commencent par le préfixe **$**
- PHP est partiellement sensible à la casse:
  - Les noms des variables sont sensibles à la casse, les noms des fonctions et les mots clés ne sont pas sensibles à la casse
- Exemple :

```
$a = 12;
$b=‘bonjour’;
```

Les variables prédéfinies

- PHP dispose d'un grand nombre de variables prédéfinies,
  - informations à la fois sur le serveur et sur toutes les données qui peuvent transiter entre le poste client et le serveur, comme les valeurs saisies dans un formulaire, les cookies ou les sessions
- Ces variables se présentent sous la forme de tableaux, accessibles en tout point de n'importe quel script. On appelle ces tableaux superglobaux.
- http://php.net/manual/fr/language.variables.superglobals.php

Les variables prédéfinies

$GLOBALS	Contient le nom et la valeur de toutes les variables globales du script. Les noms des variables sont les clés de ce tableau.
$_ENV	Contient le nom et la valeur des variables d'environnement qui sont changeantes selon les serveurs.
$_GET	Contient le nom et la valeur des données issues d'un formulaire envoyé par la méthode GET. Les noms des champs du formulaire sont les clés de ce tableau
$_POST	Contient le nom et la valeur des données issues d'un formulaire envoyé par la méthode POST. Les noms des champs du formulaire sont les clés de ce tableau.
$_REQUEST	Contient l'ensemble des variables superglobales $_GET, $_POST, $_COOKIE et $_FILES.

Les variables prédéfinies

$_SERVER	Contient les informations liées au serveur Web, tel le contenu des en-têtes HTTP ou le nom du script en cours d'exécution: **$_SERVER["HTTP_ACCEPT_LANGUAGE"]**: le code de langue du navigateur client. **$_SERVER["HTTP_COOKIE"]** : le nom et la valeur des cookies lus sur le poste client. **$_SERVER["HTTP_HOST"]**: le nom de domaine. **$_SERVER["SERVER_ADDR"]:** l'adresse IP du serveur. **$_SERVER["PHP_SELF"]:** le nom du script en cours **$_SERVER["QUERY_STRING"]:** la chaîne de la requête utilisée pour accéder au script.
$_FILES	Contient les noms des fichiers téléchargés depuis le poste du client.
$_SESSION	Contient l'ensemble des noms des variables de session et leurs valeurs.
$_COOKIE	Contient le nom et la valeur des cookies enregistrés sur le poste client. Les noms des cookies sont les clés de ce tableau

Les constantes

Les constantes

- Pour définir une constante, il suffit d'utiliser l'instruction define

```
define("nom_constante", valeur_constante, casse )
```

  - **casse** est un booleen qui détermine si le nom de la constante est sensible à la casse ou non.

- Exemple

```
<?php
   define ("MACONSTANTE", "Bonjour");
   if (defined("MACONSTANTE")) {
       Echo "La valeur de ma constante est :".MACONSTANTE;
   }
?>
```

  - **defined("nom_constante")** permet de vérifier l'existence de la constante

Les constantes

- **Quelques constantes prédéfinies**
  - NULL
  - _FILE_
  - _LINE_
  - PHP_VERSION
  - PHP_OS
  - TRUE et FALSE
- http://php.net/manual/fr/reserved.constants.php

Les types de données

Les types de variables

- En PHP, il n'existe pas de déclaration explicite de type.
- PHP supporte les types de données suivant :
- **Types scalaires de base:**
  - nombres entiers: **`integer`**
  - nombres à virgule flottante: **`double`** ou bien **`float`**
  - chaînes de caractères: **`string`**
  - Booléens: **`boolean`**
- **Types composés**
  - Tableaux: **`array`**
  - Objets: **`object`**

Les types de variables

- **Types spéciaux**
  - Le type **`null`**: est attribué à une variable non initialisée, ou à laquelle on a explicitement affecté la vakeur null
  - Le type **`resource`**: Une ressource est une variable spéciale, contenant une référence vers une ressource externe: Un fichier, une connexion à une BD, etc.
    - Les ressources sont créées et utilisées par des fonctions spéciales.
    - https://www.php.net/manual/fr/resource.php
    - https://www.php.net/manual/fr/function.get-resource-type.php

Les types de variables

- PHP détermine le type des variables et les converties (généralement) comme il faut.
- PHP inclus des fonctions permettant de **déterminer le type d'une variable**:
  - **`gettype()`** retourne une chaine de caractère contenant le type de la variable.
  - **`is_integer(), is_long() , is_double(), is_numeric() , is_string(), is_bool(), is_array(), is_null(), is_resource()`** et **`is_object()`** permettent de déterminer si le type de variable correspond.
- **Pour convertir le type d'une variable**:
  - **`$resultat=(type_desire) $nom_variable`**;
  - Ou bien utiliser la fonction settype: **`booléen settype($nom_variable, 'type_desire')`**
    - Retourne true si la conversion est possible, false sinon
  - Ou bien utiliser la fonction propre à chaque type: **`floatval(), intval(), strval()`**

Les types de variables

- PHP inclus des fonctions permettant de **vérifier l'état d'une variable**:
  - **`var_dump ():`** Affiche les informations d'une variable
  - **`empty():`** Détermine si une variable est vide
  - **`isset():`** Détermine si une variable est définie et est différente de NULL

- **`unset():`** Détruit une variable

Les opérateurs

Les opérateurs arithmétiques

- $a++ : Incrémentation
- $a -- : Décrémentation
- $a + $b : Addition : Somme de $a et $b .
- $a - $b : Soustraction : Différence de $a et $b .
- $a * $b : Multiplication: Produit de $a et $b .
- $a / $b : Division : Quotient de $a et $b .
- $a % $b : Modulo : Reste de $a divisé par $b

Les opérateurs de comparaison

- $a == $b : Egal : Vrai si $a est égal à $b.
- $a != $b : Différent : Vrai si $a est différent de $b.
- $a <> $b : Différent : Vrai si $a est différent de $b.
- $a < $b : inférieur : Vrai si $a est strictement > à $b .
- $a > $b : supérieur : Vrai si $a est strictement < à $b .
- $a <= $b : inférieur ou égal : Vrai si $a est plus petit ou égal à $b .
- $a >= $b : supérieur ou égal : Vrai si $a est plus grand ou égal à $b
- $a === $b : teste l'égalité des types et des valeurs; égale à True si $a et $b de même type et de valeur égales, False sinon
- $a !== $b : la non égalité des types et des valeurs

Les opérateurs logiques

- $a and $b : ET   : Vrai si $a ET $b sont vrais.
- $a or $b : OU   : Vrai si $a OU $b est vrai.

- $a && $b : ET   : Vrai si $a ET $b sont vrais.
- $a || $b   : OU   : Vrai si $a OU $b est vrai.

- $a xor $b : XOR : Vrai si $a OU $b est vrai, mais pas les 2.
- ! $a : NON : Vrai si $a est faux.

Les structures de contrôle

Instructions conditionnelles

```
if (conditionA) {
   // instructions A
} elseif (conditionB1 &&
conditionB2) {

   // instructions B
} elseif (conditionC1 ||
conditionC2) {

   // instructions C
} else {

   // instructions D
}
```

Instructions conditionnelles

- if then else

```
if( $var == "ok")
{
    echo « succes";
}
else{
    echo « echec";
}
```

- if else elseif

```
$variable = "voiture";
if($variable == "voiture"){
    echo " bravo vous avez trouvé ";
}
elseif($variable== "automobile"){
    echo " c'est presque ça "; }
else {
    echo " veuillez réessayer " ;}
```

Instructions conditionnelles

```
switch (Variable) {
    case Valeur1:
      //Liste d'instructions1
    break;
    case Valeur2:
      //Liste d'instructions2
    break;
    …
default: Liste d'instructions
break;
}
```

Instructions conditionnelles

- switch

```
switch($operation)
{
    case '1': // si la variable opération est égale à 1
        echo ' opération numero 1';
        break;
    case '2': // si la variable opération est égale à 2
        echo 'opération numero 2';
        break;
    default: /* si la variable opération n' est pas égale à 1 ni à 2
            ou si elle n'est pas définie */
        echo 'opération par defaut';
}
```

Boucles

```
for ($i=0 ; $i < $n ; $i++) {

   // instructions
}
```

```
while (condition) {

   // instructions
}
```

```
do {

   // instructions

} while (condition)
```

Boucles

- while

```
$i= 0;
while( $i <= 4 ) {
    echo "itération numéro $i <br> ";
    $i++;  }
```

- do

```
$i= 0;
do {
    echo "itération numéro $i <br> ";
    $i++;  } while( $i <= 4 );
```

- for

```
for( $i=0; $i <= 4; $i++ ) {
    echo "itération numéro $i <br> "; }
```

Questions?